IronHabit Lab

Database PostgreSQL popolato, con 20 query, soluzioni SQL e righe attese.

Area	Tabelle principali
Anagrafica	gyms, coaches, members
Abbonamenti	membership_plans, member_memberships, payments
Allenamento	muscle_groups, exercises, workout_plans, plan_exercises, assignments
Log e obiettivi	workout_sessions, performed_sets, goals, goal_checkins

Script SQL collegato: output/sql/ironhabit_lab_postgres.sql. Tutti i risultati attesi sono stati calcolati dagli stessi dati seed.

1. Membri attivi con palestra, coach e piano corrente

Mostra tutti i membri attivi, la loro palestra, il coach principale e il piano di allenamento assegnato oggi.

Soluzione SQL

```
SELECT
    m.member_id,
    m.first_name || ' ' || m.last_name AS member_name,
    g.city AS home_city,
    c.first_name || ' ' || c.last_name AS coach,
    wp.name AS current_plan,
    a.target_sessions_per_week AS target_sessions
FROM members m
JOIN gyms g ON g.gym_id = m.home_gym_id
JOIN coaches c ON c.coach_id = m.primary_coach_id
LEFT JOIN member_plan_assignments a
    ON a.member_id = m.member_id
    AND a.end_date IS NULL
LEFT JOIN workout_plans wp ON wp.workout_plan_id = a.workout_plan_id
WHERE m.is_active = TRUE
ORDER BY m.member_id;
```

Risultato atteso - 9 righe

member_id	member_name	home_city	coach	current_plan	target_sessions
1	Giulia Rossi	Roma	Sara Conti	Hypertrophy Split	4
2	Andrea Marino	Roma	Luca Bianchi	Strength Starter	3
3	Fatima El Amrani	Milano	Marco Russo	Fat Loss Circuit	4
4	Paolo Verdi	Milano	Marco Russo	Powerlifting Prep	5
5	Martina Gallo	Napoli	Elena Ferrari	Mobility and Core	3
6	Chen Wei	Roma	Luca Bianchi	Strength Starter	3
8	Omar Haddad	Napoli	Elena Ferrari	Fat Loss Circuit	4
9	Sofia Esposito	Roma	Sara Conti	Hypertrophy Split	3
10	Davide Ricci	Roma	Luca Bianchi	Powerlifting Prep	4

2. Abbonamenti attivi in scadenza

Trova gli abbonamenti attivi che scadono tra il 1 giugno 2026 e il 15 luglio 2026.

Soluzione SQL

```
SELECT
    m.first_name || ' ' || m.last_name AS member_name,
    mp.plan_name,
    mm.end_date,
    mm.status
FROM member_memberships mm
JOIN members m ON m.member_id = mm.member_id
JOIN membership_plans mp ON mp.plan_id = mm.plan_id
WHERE mm.status = 'active'
  AND mm.end_date BETWEEN DATE '2026-06-01' AND DATE '2026-07-15'
ORDER BY mm.end_date, member_name;
```

Risultato atteso - 7 righe

member_name	plan_name	end_date	status
Fatima El Amrani	Student	2026-06-30	active
Giulia Rossi	Plus	2026-06-30	active
Martina Gallo	Plus	2026-06-30	active
Omar Haddad	Plus	2026-06-30	active
Paolo Verdi	Annual Pro	2026-06-30	active
Sofia Esposito	Student	2026-06-30	active
Chen Wei	Plus	2026-07-14	active

3. Ricavi pagati per piano

Calcola il fatturato pagato per tipo di abbonamento nel primo semestre 2026, escludendo rimborsi e servizi extra.

Soluzione SQL

```
SELECT
    mp.plan_name,
    COUNT(p.payment_id) AS payments,
    ROUND(SUM(p.amount), 2) AS revenue
FROM payments p
JOIN member_memberships mm ON mm.membership_id = p.membership_id
JOIN membership_plans mp ON mp.plan_id = mm.plan_id
WHERE p.status = 'paid'
  AND p.paid_on BETWEEN DATE '2026-01-01' AND DATE '2026-06-30'
GROUP BY mp.plan_name
ORDER BY revenue DESC;
```

Risultato atteso - 4 righe

plan_name	payments	revenue
Annual Pro	3	1497.00
Plus	14	838.60
Student	7	209.30
Base	1	39.90

4. Ricavi mensili

Raggruppa tutti i pagamenti riusciti per mese, includendo anche servizi extra non legati a un abbonamento.

Soluzione SQL

```
SELECT
    TO_CHAR(p.paid_on, 'YYYY-MM') AS month,
    ROUND(SUM(p.amount), 2) AS revenue
FROM payments p
WHERE p.status = 'paid'
  AND p.paid_on BETWEEN DATE '2026-01-01' AND DATE '2026-06-30'
GROUP BY TO_CHAR(p.paid_on, 'YYYY-MM')
ORDER BY month;
```

Risultato atteso - 6 righe

month	revenue
2026-01	1556.90
2026-02	159.80
2026-03	189.60
2026-04	209.60
2026-05	329.50
2026-06	419.40

5. Top 5 esercizi per volume

Trova i 5 esercizi con piu volume totale in kg nel mese di giugno 2026.

Soluzione SQL

```
SELECT
    e.name AS exercise,
    ROUND(SUM(ps.weight_kg * ps.reps), 2) AS total_volume_kg,
    COUNT(*) AS sets
FROM performed_sets ps
JOIN workout_sessions ws ON ws.session_id = ps.session_id
JOIN exercises e ON e.exercise_id = ps.exercise_id
WHERE ws.session_date BETWEEN DATE '2026-06-01' AND DATE '2026-06-30'
GROUP BY e.name
ORDER BY total_volume_kg DESC, exercise
LIMIT 5;
```

Risultato atteso - 5 righe

exercise	total_volume_kg	sets
Leg Press	21030.00	18
Squat	16195.00	30
Bench Press	15267.50	42
Rowing Machine	10200.00	22
Deadlift	10050.00	18

6. Volume per atleta

Calcola sessioni e volume totale per ogni atleta che si e allenato in giugno 2026.

Soluzione SQL

```
SELECT
    m.first_name || ' ' || m.last_name AS member_name,
    COUNT(DISTINCT ws.session_id) AS sessions,
    ROUND(SUM(ps.weight_kg * ps.reps), 2) AS volume_kg
FROM members m
JOIN workout_sessions ws ON ws.member_id = m.member_id
JOIN performed_sets ps ON ps.session_id = ws.session_id
WHERE ws.session_date BETWEEN DATE '2026-06-01' AND DATE '2026-06-30'
GROUP BY m.member_id, m.first_name, m.last_name
ORDER BY volume_kg DESC;
```

Risultato atteso - 9 righe

member_name	sessions	volume_kg
Paolo Verdi	3	17180.00
Davide Ricci	3	17070.00
Fatima El Amrani	3	15645.00
Omar Haddad	2	11940.00
Giulia Rossi	3	11570.00
Andrea Marino	2	4845.00
Martina Gallo	2	4353.00
Chen Wei	2	4267.50
Sofia Esposito	1	2138.00

7. Performance media dei piani

Per ogni piano usato a giugno, mostra quante sessioni ci sono state e le medie di durata e RPE percepito.

Soluzione SQL

```
SELECT
    wp.name AS workout_plan,
    COUNT(ws.session_id) AS session_count,
    ROUND(AVG(ws.duration_min), 1) AS avg_duration_min,
    ROUND(AVG(ws.perceived_rpe), 1) AS avg_rpe
FROM workout_sessions ws
JOIN workout_plans wp ON wp.workout_plan_id = ws.workout_plan_id
WHERE ws.session_date BETWEEN DATE '2026-06-01' AND DATE '2026-06-30'
GROUP BY wp.name
ORDER BY avg_rpe DESC, workout_plan;
```

Risultato atteso - 5 righe

workout_plan	session_count	avg_duration_min	avg_rpe
Powerlifting Prep	6	88.7	8.9
Fat Loss Circuit	5	49.0	8.0
Hypertrophy Split	4	70.3	7.6
Strength Starter	4	58.8	7.3
Mobility and Core	2	41.0	5.8

8. Carico coach e aderenza

Per ogni coach, misura membri attivi assegnati, sessioni di giugno e aderenza media rispetto al target di 4 settimane.

Soluzione SQL

```
WITH june_counts AS (
    SELECT member_id, COUNT(*) AS sessions_done
    FROM workout_sessions
    WHERE session_date BETWEEN DATE '2026-06-01' AND DATE '2026-06-30'
    GROUP BY member_id
)
SELECT
    c.first_name || ' ' || c.last_name AS coach,
    COUNT(m.member_id) AS active_members,
    SUM(COALESCE(jc.sessions_done, 0)) AS june_sessions,
    ROUND(AVG(COALESCE(jc.sessions_done, 0) * 100.0
        / (a.target_sessions_per_week * 4)), 1) AS avg_adherence_pct
FROM coaches c
JOIN members m
    ON m.primary_coach_id = c.coach_id
    AND m.is_active = TRUE
JOIN member_plan_assignments a
    ON a.member_id = m.member_id
    AND a.end_date IS NULL
LEFT JOIN june_counts jc ON jc.member_id = m.member_id
GROUP BY c.coach_id, c.first_name, c.last_name
ORDER BY active_members DESC, coach;
```

Risultato atteso - 4 righe

coach	active_members	june_sessions	avg_adherence_pct
Luca Bianchi	3	7	17.4
Elena Ferrari	2	4	14.6
Marco Russo	2	6	16.9
Sara Conti	2	4	13.5

9. Miglior carico sui big lift

Per squat, bench press e deadlift, mostra il miglior carico registrato da ogni atleta.

Soluzione SQL

```
SELECT
    m.first_name || ' ' || m.last_name AS member_name,
    e.name AS exercise,
    MAX(ps.weight_kg) AS max_weight_kg
FROM performed_sets ps
JOIN workout_sessions ws ON ws.session_id = ps.session_id
JOIN members m ON m.member_id = ws.member_id
JOIN exercises e ON e.exercise_id = ps.exercise_id
WHERE e.name IN ('Bench Press', 'Squat', 'Deadlift')
GROUP BY m.member_id, m.first_name, m.last_name, e.name
ORDER BY e.name, max_weight_kg DESC, member_name;
```

Risultato atteso - 15 righe

member_name	exercise	max_weight_kg
Paolo Verdi	Bench Press	105
Davide Ricci	Bench Press	95
Andrea Marino	Bench Press	60
Chen Wei	Bench Press	52.50
Giulia Rossi	Bench Press	50
Sofia Esposito	Bench Press	35
Laura Neri	Bench Press	30
Paolo Verdi	Deadlift	180
Davide Ricci	Deadlift	170
Andrea Marino	Deadlift	120
Chen Wei	Deadlift	110
Paolo Verdi	Squat	147.50
Davide Ricci	Squat	145
Andrea Marino	Squat	95
Chen Wei	Squat	80

10. Classifica sessioni 2026

Crea una classifica dei membri per numero di sessioni registrate nel 2026.

Soluzione SQL

```
WITH counts AS (
    SELECT
        m.member_id,
        m.first_name || ' ' || m.last_name AS member_name,
        COUNT(ws.session_id) AS sessions_2026
    FROM members m
    LEFT JOIN workout_sessions ws
        ON ws.member_id = m.member_id
        AND ws.session_date BETWEEN DATE '2026-01-01' AND DATE '2026-12-31'
    GROUP BY m.member_id, m.first_name, m.last_name
)
SELECT
    DENSE_RANK() OVER (ORDER BY sessions_2026 DESC) AS ranking,
    member_name,
    sessions_2026
FROM counts
ORDER BY ranking, member_name
LIMIT 10;
```

Risultato atteso - 10 righe

ranking	member_name	sessions_2026
1	Fatima El Amrani	5
1	Giulia Rossi	5
2	Andrea Marino	4
2	Davide Ricci	4
2	Paolo Verdi	4
3	Chen Wei	3
3	Omar Haddad	3
4	Martina Gallo	2
4	Sofia Esposito	2
5	Laura Neri	1

11. Obiettivi sotto il 70 percento

Prendi l'ultimo check-in di ogni obiettivo attivo e mostra quelli sotto il 70 percento di progresso.

Soluzione SQL

```
WITH latest_checkin AS (
    SELECT
        gc.*,
        ROW_NUMBER() OVER (
            PARTITION BY gc.goal_id
            ORDER BY gc.checked_on DESC
        ) AS rn
    FROM goal_checkins gc
),
progress AS (
    SELECT
        m.first_name || ' ' || m.last_name AS member_name,
        g.goal_type,
        g.target_value,
        lc.current_value,
        ROUND(lc.current_value * 100.0 / g.target_value, 1) AS progress_pct,
        g.due_date
    FROM goals g
    JOIN members m ON m.member_id = g.member_id
    JOIN latest_checkin lc ON lc.goal_id = g.goal_id AND lc.rn = 1
    WHERE g.status = 'active'
)
SELECT *
FROM progress
WHERE progress_pct < 70
ORDER BY progress_pct, member_name;
```

Risultato atteso - 4 righe

member_name	goal_type	target_value	current_value	progress_pct	due_date
Fatima El Amrani	monthly_sessions	16	7	43.8	2026-06-30
Omar Haddad	weight_loss	4	2.2	55.0	2026-07-31
Chen Wei	monthly_sessions	12	8	66.7	2026-06-30
Martina Gallo	plank_seconds	180	120	66.7	2026-07-31

12. Esercizi mai eseguiti

Trova gli esercizi presenti nel catalogo ma mai usati in nessun set registrato.

Soluzione SQL

```
SELECT
    e.exercise_id,
    e.name AS exercise,
    mg.name AS muscle_group
FROM exercises e
JOIN muscle_groups mg ON mg.muscle_group_id = e.muscle_group_id
LEFT JOIN performed_sets ps ON ps.exercise_id = e.exercise_id
WHERE ps.set_id IS NULL
ORDER BY e.exercise_id;
```

Risultato atteso - 1 righe

exercise_id	exercise	muscle_group
13	Battle Rope	Full Body

13. Piani: esercizi e utenti

Per ogni piano di allenamento, conta esercizi distinti previsti e membri attivi attualmente assegnati.

Soluzione SQL

```
SELECT
    wp.name AS workout_plan,
    COUNT(DISTINCT pe.exercise_id) AS planned_exercises,
    COUNT(DISTINCT CASE
        WHEN a.end_date IS NULL AND m.is_active = TRUE THEN a.member_id
    END) AS active_assigned_members
FROM workout_plans wp
LEFT JOIN plan_exercises pe ON pe.workout_plan_id = wp.workout_plan_id
LEFT JOIN member_plan_assignments a ON a.workout_plan_id = wp.workout_plan_id
LEFT JOIN members m ON m.member_id = a.member_id
GROUP BY wp.workout_plan_id, wp.name
ORDER BY active_assigned_members DESC, workout_plan;
```

Risultato atteso - 5 righe

workout_plan	planned_exercises	active_assigned_members
Fat Loss Circuit	4	2
Hypertrophy Split	5	2
Powerlifting Prep	4	2
Strength Starter	6	2
Mobility and Core	3	1

14. Atleti sotto target sessioni

Mostra gli atleti attivi che in giugno 2026 hanno fatto meno sessioni del target mensile stimato.

Soluzione SQL

```
WITH june_counts AS (
    SELECT member_id, COUNT(*) AS done_sessions
    FROM workout_sessions
    WHERE session_date BETWEEN DATE '2026-06-01' AND DATE '2026-06-30'
    GROUP BY member_id
)
SELECT
    m.first_name || ' ' || m.last_name AS member_name,
    a.target_sessions_per_week * 4 AS target_sessions,
    COALESCE(jc.done_sessions, 0) AS done_sessions,
    a.target_sessions_per_week * 4 - COALESCE(jc.done_sessions, 0) AS missing_sessions
FROM members m
JOIN member_plan_assignments a
    ON a.member_id = m.member_id
    AND a.end_date IS NULL
LEFT JOIN june_counts jc ON jc.member_id = m.member_id
WHERE m.is_active = TRUE
  AND COALESCE(jc.done_sessions, 0) < a.target_sessions_per_week * 4
ORDER BY missing_sessions DESC, member_name;
```

Risultato atteso - 9 righe

member_name	target_sessions	done_sessions	missing_sessions
Paolo Verdi	20	3	17
Omar Haddad	16	2	14
Davide Ricci	16	3	13
Fatima El Amrani	16	3	13
Giulia Rossi	16	3	13
Sofia Esposito	12	1	11
Andrea Marino	12	2	10
Chen Wei	12	2	10
Martina Gallo	12	2	10

15. Carico medio per esercizio

Per giugno 2026, calcola il carico medio dei set con peso maggiore di zero, raggruppato per muscolo ed esercizio.

Soluzione SQL

```
SELECT
    mg.name AS muscle_group,
    e.name AS exercise,
    ROUND(AVG(ps.weight_kg), 1) AS avg_load_kg,
    COUNT(*) AS weighted_sets
FROM performed_sets ps
JOIN workout_sessions ws ON ws.session_id = ps.session_id
JOIN exercises e ON e.exercise_id = ps.exercise_id
JOIN muscle_groups mg ON mg.muscle_group_id = e.muscle_group_id
WHERE ws.session_date BETWEEN DATE '2026-06-01' AND DATE '2026-06-30'
  AND ps.weight_kg > 0
GROUP BY mg.name, e.name
HAVING COUNT(*) >= 3
ORDER BY mg.name, e.name;
```

Risultato atteso - 10 righe

muscle_group	exercise	avg_load_kg	weighted_sets
Arms	Biceps Curl	10.00	9
Back	Deadlift	151.70	18
Cardio	Rowing Machine	33.60	22
Chest	Bench Press	76.10	42
Chest	Cable Fly	15.00	9
Legs	Hip Thrust	61.70	9
Legs	Leg Press	103.30	18
Legs	Romanian Deadlift	107.50	6
Legs	Squat	131.20	30
Shoulders	Shoulder Press	28.30	12

16. Spesa sopra la media

Trova i membri che hanno speso piu della media dei totali pagati per membro.

Soluzione SQL

```
WITH totals AS (
    SELECT member_id, ROUND(SUM(amount), 2) AS total_spent
    FROM payments
    WHERE status = 'paid'
    GROUP BY member_id
),
average_total AS (
    SELECT AVG(total_spent) AS avg_spent
    FROM totals
)
SELECT
    m.first_name || ' ' || m.last_name AS member_name,
    t.total_spent
FROM totals t
JOIN members m ON m.member_id = t.member_id
CROSS JOIN average_total a
WHERE t.total_spent > a.avg_spent
ORDER BY t.total_spent DESC, member_name;
```

Risultato atteso - 4 righe

member_name	total_spent
Davide Ricci	619.00
Paolo Verdi	589.00
Andrea Marino	569.00
Giulia Rossi	359.40

17. Varieta compound

Mostra gli atleti che in giugno hanno eseguito almeno 3 esercizi compound diversi.

Soluzione SQL

```
SELECT
    m.first_name || ' ' || m.last_name AS member_name,
    COUNT(DISTINCT e.exercise_id) AS compound_exercises
FROM members m
JOIN workout_sessions ws ON ws.member_id = m.member_id
JOIN performed_sets ps ON ps.session_id = ws.session_id
JOIN exercises e ON e.exercise_id = ps.exercise_id
WHERE ws.session_date BETWEEN DATE '2026-06-01' AND DATE '2026-06-30'
  AND e.is_compound = TRUE
GROUP BY m.member_id, m.first_name, m.last_name
HAVING COUNT(DISTINCT e.exercise_id) >= 3
ORDER BY compound_exercises DESC, member_name;
```

Risultato atteso - 5 righe

member_name	compound_exercises
Andrea Marino	5
Chen Wei	5
Davide Ricci	4
Paolo Verdi	4
Giulia Rossi	3

18. Miglioramento Bench Press

Confronta la prima e l'ultima sessione di Bench Press di ogni atleta e mostra solo chi e migliorato.

Soluzione SQL

```
WITH bench_by_day AS (
    SELECT
        ws.member_id,
        ws.session_date,
        MAX(ps.weight_kg) AS max_weight
    FROM workout_sessions ws
    JOIN performed_sets ps ON ps.session_id = ws.session_id
    JOIN exercises e ON e.exercise_id = ps.exercise_id
    WHERE e.name = 'Bench Press'
    GROUP BY ws.member_id, ws.session_date
),
ranked AS (
    SELECT
        *,
        ROW_NUMBER() OVER (
            PARTITION BY member_id ORDER BY session_date
        ) AS first_rank,
        ROW_NUMBER() OVER (
            PARTITION BY member_id ORDER BY session_date DESC
        ) AS last_rank
    FROM bench_by_day
),
pivoted AS (
    SELECT
        member_id,
        MAX(CASE WHEN first_rank = 1 THEN max_weight END) AS first_weight,
        MAX(CASE WHEN last_rank = 1 THEN max_weight END) AS latest_weight
    FROM ranked
    GROUP BY member_id
)
SELECT
    m.first_name || ' ' || m.last_name AS member_name,
    first_weight,
    latest_weight,
    ROUND(latest_weight - first_weight, 2) AS improvement_kg
FROM pivoted p
JOIN members m ON m.member_id = p.member_id
WHERE latest_weight > first_weight
ORDER BY improvement_kg DESC, member_name;
```

Risultato atteso - 6 righe

member_name	first_weight	latest_weight	improvement_kg
Giulia Rossi	40	50	10
Davide Ricci	87.50	95	7.5
Andrea Marino	55	60	5
Paolo Verdi	100	105	5
Chen Wei	50	52.50	2.5
Sofia Esposito	32.50	35	2.5

19. Migliore sessione per volume

Per ogni membro attivo, individua la sessione con il volume piu alto e mostra le prime 8.

Soluzione SQL

```
WITH session_volume AS (
    SELECT
        ws.session_id,
        ws.member_id,
        ws.session_date,
        ROUND(SUM(ps.weight_kg * ps.reps), 2) AS volume_kg
    FROM workout_sessions ws
    JOIN performed_sets ps ON ps.session_id = ws.session_id
    GROUP BY ws.session_id, ws.member_id, ws.session_date
),
ranked AS (
    SELECT
        *,
        ROW_NUMBER() OVER (
            PARTITION BY member_id
            ORDER BY volume_kg DESC, session_date DESC
        ) AS rn
    FROM session_volume
)
SELECT
    m.first_name || ' ' || m.last_name AS member_name,
    r.session_date,
    r.volume_kg
FROM ranked r
JOIN members m ON m.member_id = r.member_id
WHERE r.rn = 1
  AND m.is_active = TRUE
ORDER BY r.volume_kg DESC, member_name
LIMIT 8;
```

Risultato atteso - 8 righe

member_name	session_date	volume_kg
Omar Haddad	2026-06-10	6900.00
Paolo Verdi	2026-06-07	6600.00
Davide Ricci	2026-06-01	6420.00
Giulia Rossi	2026-06-06	6420.00
Fatima El Amrani	2026-06-05	5250.00
Martina Gallo	2026-06-11	2580.00
Andrea Marino	2026-06-09	2520.00
Chen Wei	2026-06-16	2280.00

20. Stato abbonamenti per piano

Riepiloga quanti abbonamenti sono mai esistiti per piano e quanti risultano attivi o non attivi.

Soluzione SQL

```
SELECT
    mp.plan_name,
    COUNT(mm.membership_id) AS memberships_total,
    SUM(CASE WHEN mm.status = 'active' THEN 1 ELSE 0 END) AS active_memberships,
    SUM(CASE WHEN mm.status <> 'active' THEN 1 ELSE 0 END) AS inactive_memberships
FROM membership_plans mp
LEFT JOIN member_memberships mm ON mm.plan_id = mp.plan_id
GROUP BY mp.plan_id, mp.plan_name
ORDER BY memberships_total DESC, mp.plan_name;
```

Risultato atteso - 5 righe

plan_name	memberships_total	active_memberships	inactive_memberships
Plus	4	4	0
Annual Pro	3	3	0
Base	2	0	2
Student	2	2	0
Drop-in	0	0	0